```
public IActionResult OnPost(UserBindingModel user)
```

Alternatively, using the `[BindProperty]` approach, create a property on the `PageModel`:

```
[BindProperty]
public UserBindingModel User { get; set; }
```

Now you can simplify the page handler signature even further:

```
public IActionResult OnPost()
```

Functionally, the model binder treats this new complex type a little differently. Rather than look for parameters with a value that matches the parameter name (`user`, or `User` for the property), the model binder creates a new instance of the model using `new UserBindingModel()`.

NOTE You don't have to use custom classes for your methods; it depends on your requirements. If your page handler needs only a single integer, it makes more sense to bind to the simple parameter.

Next, the model binder loops through all the properties your binding model has, such as `FirstName` and `LastName` in listing 16.4. For each of these properties, it consults the collection of binding sources and attempts to find a name-value pair that matches. If it finds one, it sets the value on the property and moves on to the next.

TIP Although the name of the model isn't necessary in this example, the model binder will also look for properties prefixed with the name of the property, such as `user.FirstName` and `user.LastName` for a property called `User`. You can use this approach when you have multiple complex parameters to a page handler or multiple complex `[BindProperty]`